

短期电力负荷预测：数据处理与特征构建

本次只汇报数据处理部分：原始数据理解、清洗重构、缺失值判断、2014 年负荷分析、天气合并、建模特征表构建。算法训练和模型选择放到第二次汇报。

1. 原始数据理解

附件 Excel 包含两个核心工作表：**Area_Load** 是电力负荷，**Area_Weather** 是天气。负荷表中每天有 96 个 15 分钟采样点，因此先要把宽表转换成可分析的日粒度数据。

检查项	结果	说明
Area_Load 原始负荷表	2208 行 x 97 列	每天一行，96 个 15 分钟采样点。
Area_Weather 原始天气表	1113 行 x 6 列	每天一行，包含最高/最低/平均温度、湿度、降雨量。
负荷采样列数	96 个	$96 = 24 \text{ 小时} \times \text{每小时 } 4 \text{ 个 } 15 \text{ 分钟点}$ 。
预测目标期负荷缺失	672 个单元格	正好 7 天 x 96 点，是题目要求预测的目标，不作为异常删除。
目标期天气是否完整	0 个缺失	递推预测时可以使用目标期已给定天气。
建模日粒度表	2208 行 x 23 列	由 15 分钟负荷聚合为日最大、日最小、日平均。

Area_Load 原始表截图

Area_Load 原始负荷表节选

YMD	T0000	T0015	T0030	T0045	T0100	T0115	T0130	T0145	T0200	T0215	T2330	T2345
20090101	3210.84	3104.51	3037.71	2960.18	2889.23	2827.70	2772.51	2720.47	2666.59	2632.01	2382.70	2276.21
20090102	2163.99	2074.74	1999.62	1950.29	1889.14	1846.49	1800.41	1778.31	1746.28	1716.47	3424.57	3266.95
20090103	3109.13	3015.90	2938.31	2869.38	2815.77	2777.30	2725.38	2704.33	2665.86	2644.08	3602.28	3444.15
20090104	3307.75	3209.73	3125.51	3075.18	3028.71	2995.46	2956.24	2926.63	2893.67	2870.43	3602.40	3477.08
20090105	3295.42	3278.38	3232.31	3160.73	3124.23	3092.29	3064.08	3034.58	3007.34	2984.58	4106.36	3933.51
20090106	3780.43	3678.76	3583.49	3541.04	3473.69	3422.26	3368.35	3343.13	3304.07	3274.83	4111.92	3936.80
20090107	3746.87	3677.80	3608.53	3562.01	3504.42	3453.95	3412.27	3389.51	3350.03	3334.94	4113.67	3955.63
20090108	3786.99	3662.89	3593.44	3542.98	3484.11	3453.87	3414.92	3385.65	3342.56	3323.98	4116.19	3928.63

Area_Weather 原始表截图

Area_Weather 原始天气表节选

YMD	temp_max	temp_min	temp_avg	humidity	rainfall
20120101	19.50	12.10	15.80	63.00	0.00
20120102	20.00	13.00	16.00	59.00	0.00
20120103	18.70	14.20	15.80	72.00	0.00
20120104	14.90	9.30	10.90	62.00	0.00
20120105	9.20	5.10	6.90	78.00	2.90
20120106	11.00	6.30	8.20	92.00	3.30
20120107	12.40	9.40	10.50	80.00	0.00
20120108	16.60	9.90	12.20	75.00	0.00

2. 数据处理主线

处理流程从原始 Excel 出发，先重构负荷表，再聚合为题目要求预测的日最高、日最低、日平均负荷，并与天气表合并。

数据处理主线：从 15 分钟负荷宽表到可建模的日粒度特征表



关键原则：只用预测日前已经知道的信息构造滞后/滚动特征；2015-01-11 至 2015-01-17 的负荷缺失是预测目标。

讲解重点：为什么不是直接把 96 个时刻都拿去预测？因为项目目标是预测 2015-01-11 至 2015-01-17 的日最高、日最低、日平均负荷，日粒度聚合能和目标完全对应。

3. 代码截图：读取、重构、聚合、合并

核心代码 1：读取、重构、聚合、合并

```

def load_daily_data() -> pd.DataFrame:
    if not DATA_PATH.exists():
        raise FileNotFoundError(f"Data file not found: {DATA_PATH}")

    # 原始负荷表是“每天一行、96个15分钟采样点”的宽表；建模前先压缩成日粒度。
    # 这样能直接对应题目要求的日最高、日最低、日平均三个预测目标。
    load_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Load")
    weather_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Weather")
    weather_raw.columns = ["YMD", "temp_max", "temp_min", "temp_avg", "humidity", "rainfall"]

    time_cols = [col for col in load_raw.columns if col != "YMD"]
    long_df = load_raw.melt(id_vars="YMD", value_vars=time_cols, var_name="time_slot", value_name="load")
    daily = (
        long_df.groupby("YMD", as_index=False)
        .agg(load_max=("load", "max"), load_min=("load", "min"), load_mean=("load", "mean"))
        .merge(weather_raw, on="YMD", how="left")
    )
    daily["date"] = pd.to_datetime(daily["YMD"].astype(str), format="%Y%m%d")
    return add_features(daily.sort_values("date").reset_index(drop=True))
  
```

推导解释

原始负荷宽表的每一行是一日，列 T0000 到 T2345 表示一天内 96 个采样点。使用 `melt` 将宽表变成长表后，每条记录对应“某一天某一时刻的负荷”；再按 YMD 分组，分别取最大值、最小值和平均值，得到三个预测目标。

```

def load_daily_data() -> pd.DataFrame:
    if not DATA_PATH.exists():
        raise FileNotFoundError(f"Data file not found: {DATA_PATH}")
  
```

```

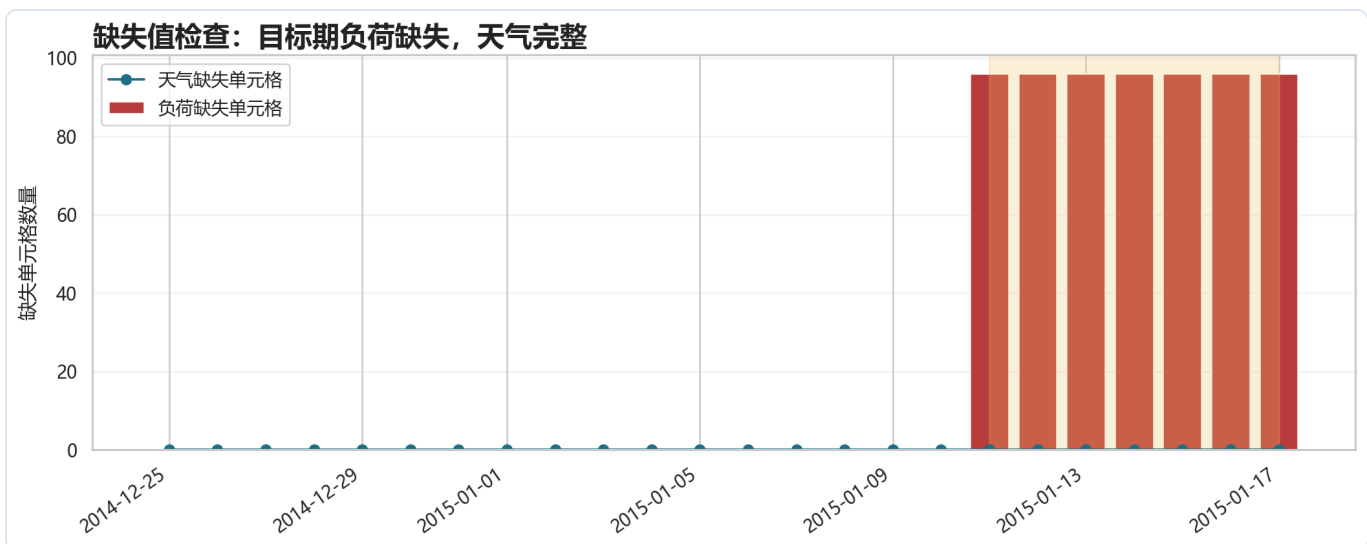
# 原始负荷表是“每天一行、96个15分钟采样点”的宽表；建模前先压缩成日粒度。
# 这样能直接对应题目要求的日最高、日最低、日平均三个预测目标。
load_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Load")
weather_raw = pd.read_excel(DATA_PATH, sheet_name="Area_Weather")
weather_raw.columns = ["YMD", "temp_max", "temp_min", "temp_avg", "humidity",
"rainfall"]

time_cols = [col for col in load_raw.columns if col != "YMD"]
long_df = load_raw.melt(id_vars="YMD", value_vars=time_cols, var_name="time_slot",
value_name="load")
daily = (
    long_df.groupby("YMD", as_index=False)
        .agg(load_max=("load", "max"), load_min=("load", "min"), load_mean=("load",
"mean"))
        .merge(weather_raw, on="YMD", how="left")
)
daily["date"] = pd.to_datetime(daily["YMD"].astype(str), format="%Y%m%d")
return add_features(daily.sort_values("date").reset_index(drop=True))

```

4. 缺失值检查与业务判断

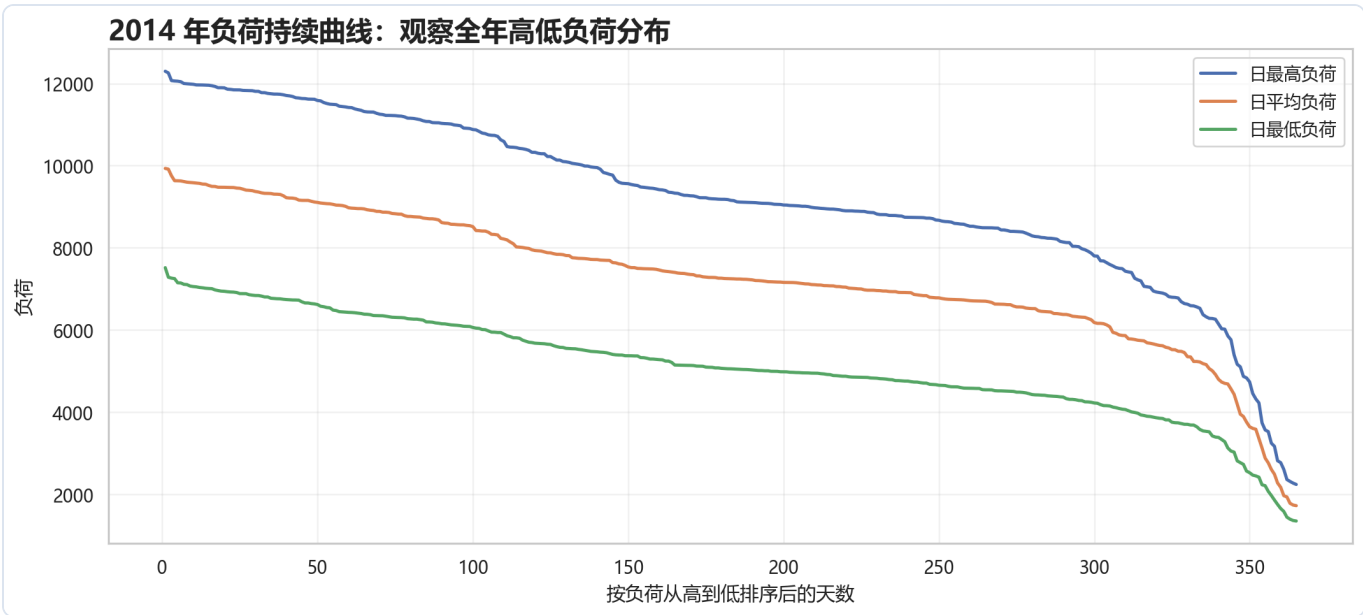
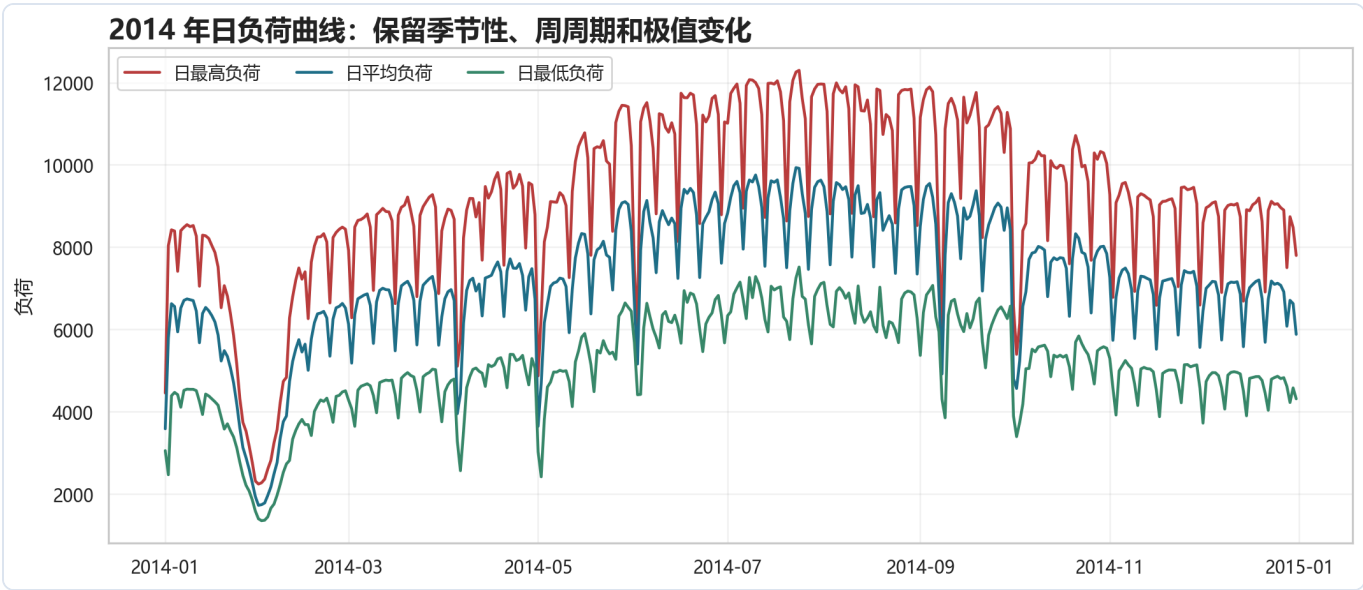
2015-01-11 至 2015-01-17 的负荷缺失不是脏数据，而是题目要求预测的未来目标；天气数据在这一段完整，可以作为预测输入。



结论：不删除目标期 7 天，也不使用插值填补目标负荷；后续预测要用递推方式生成这 7 天的负荷。

5. 2014 年负荷分析

按照实训要求，对 2014 年的日负荷进行分析。日最高、日平均、日最低三条曲线可以展示季节性和短期波动；负荷持续曲线可以展示全年高负荷和低负荷分布。



6. 天气合并与衍生变量

天气表按 YMD 与日负荷表合并。进一步构造 temp_range、hdd、cdd，让模型能表达“低温供热”和“高温制冷”对负荷的影响。

日粒度负荷 + 天气 + 衍生变量节选

YMD	load_max	load_min	load_mean	temp_max	temp_min	temp_avg	humidity	rainfall	temp_range	hdd	cdd
20090101	3210.84	2122.43	2573.97								
20090102	4865.44	1616.35	3479.02								
20090103	5183.23	2489.03	4026.50								
20090104	5164.95	2679.78	4076.88								
20090105	5852.40	2822.75	4548.92								
20090106	5909.08	3082.16	4680.46								
20090107	5816.91	3117.07	4675.43								
20090108	5867.26	3133.23	4668.18								
20090109	5893.13	3156.11	4697.20								
20090110	5550.50	3092.10	4400.61								

7. 代码截图：日历、周期和天气特征

核心代码 2：日历周期与天气衍生特征

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["dayofweek"] = df["date"].dt.dayofweek
    df["month"] = df["date"].dt.month
    df["day_of_year"] = df["date"].dt.dayofyear
    df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)

    # 周期特征不能直接用“月份=12、1”这种数值距离，因为12月和1月在时间上相邻。
    # sin/cos 编码把周期映射到圆上，保留“首尾相接”的季节/星期规律。
    df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
    df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
    df["dow_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
    df["dow_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)
    df["doy_sin"] = np.sin(2 * np.pi * df["day_of_year"] / 365)
    df["doy_cos"] = np.cos(2 * np.pi * df["day_of_year"] / 365)

    # HDD/CDD 是电力负荷常用气象衍生变量：低温带来供热需求，高温带来制冷需求。
    df["temp_range"] = df["temp_max"] - df["temp_min"]
    df["hdd"] = np.maximum(18 - df["temp_avg"], 0)
    df["cdd"] = np.maximum(df["temp_avg"] - 26, 0)
    return df
```

推导解释

月份、星期和年内日序号都是周期变量。如果直接把 12 月和 1 月当普通数字，模型会误以为二者距离很远；所以使用 sin/cos 编码，把周期映射到圆上，保留首尾相接的季节规律。

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["dayofweek"] = df["date"].dt.dayofweek
    df["month"] = df["date"].dt.month
    df["day_of_year"] = df["date"].dt.dayofyear
    df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)

    # 周期特征不能直接用“月份=12、1”这种数值距离，因为12月和1月在时间上相邻。
    # sin/cos 编码把周期映射到圆上，保留“首尾相接”的季节/星期规律。
    df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
    df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
    df["dow_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
    df["dow_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)
    df["doy_sin"] = np.sin(2 * np.pi * df["day_of_year"] / 365)
    df["doy_cos"] = np.cos(2 * np.pi * df["day_of_year"] / 365)

    # HDD/CDD 是电力负荷常用气象衍生变量：低温带来供热需求，高温带来制冷需求。
    df["temp_range"] = df["temp_max"] - df["temp_min"]
    df["hdd"] = np.maximum(18 - df["temp_avg"], 0)
    df["cdd"] = np.maximum(df["temp_avg"] - 26, 0)
    return df
```

8. 代码截图：滞后与滚动特征

特征工程结构：让模型同时看到天气、日历和历史负荷惯性



说明：滚动统计使用 shift(1) 后再 rolling，表示今天预测时只使用昨天及以前的真实历史，避免数据泄漏。

核心代码 3：滞后与滚动特征，避免数据泄漏

```
def make_target_features(df: pd.DataFrame, target: str) -> tuple[pd.DataFrame, list[str]]:
    out = df.copy()
    lag_features: list[str] = []

    # 负荷序列有强惯性和周周期：昨天、上周同日、两周前同日通常是强预测信号。
    # 每个目标单独构造滞后项，避免用“日均负荷”的历史去预测“日最高/最低”时混淆目标。
    for lag in [1, 7, 14]:
        col = f"{target}_lag_{lag}"
        out[col] = out[target].shift(lag)
        lag_features.append(col)

    # 滚动均值/标准差描述近期负荷水平和波动程度，是短期预测里很实用的平滑特征。
    for win in [7, 14]:
        mean_col = f"{target}_roll_mean_{win}"
        std_col = f"{target}_roll_std_{win}"
        out[mean_col] = out[target].shift(1).rolling(win).mean()
        out[std_col] = out[target].shift(1).rolling(win).std()
        lag_features.extend([mean_col, std_col])
    return out, MODEL_WEATHER + CALENDAR_FEATURES + lag_features
```

推导解释

电力负荷有明显惯性：昨天、上周同日、两周前同日通常是强信号。滚动均值和标准差用于描述近期水平和波动。所有滚动统计都先 `shift(1)`，保证今天的预测不会偷看今天真实负荷，避免数据泄漏。

```
def make_target_features(df: pd.DataFrame, target: str) -> tuple[pd.DataFrame, list[str]]:
    out = df.copy()
    lag_features: list[str] = []

    # 负荷序列有强惯性和周周期：昨天、上周同日、两周前同日通常是强预测信号。
    # 每个目标单独构造滞后项，避免用“日均负荷”的历史去预测“日最高/最低”时混淆目标。
    for lag in [1, 7, 14]:
        col = f"{target}_lag_{lag}"
        out[col] = out[target].shift(lag)
        lag_features.append(col)

    # 滚动均值/标准差描述近期负荷水平和波动程度，是短期预测里很实用的平滑特征。
    for win in [7, 14]:
        mean_col = f"{target}_roll_mean_{win}"
        std_col = f"{target}_roll_std_{win}"
        out[mean_col] = out[target].shift(1).rolling(win).mean()
        out[std_col] = out[target].shift(1).rolling(win).std()
        lag_features.extend([mean_col, std_col])
    return out, MODEL_WEATHER + CALENDAR_FEATURES + lag_features
```


9. 第一次汇报结论

经过数据处理，原始 Excel 已转换为可建模的日粒度特征表：目标变量是日最高、日最低、日平均负荷；输入变量包含天气、日历周期、天气衍生变量、历史滞后和滚动统计。下一次汇报可以在这个数据集基础上进入算法模型比较、训练过程和最终模型选择。